

2110531 Data Science and Data Engineering Tools (2026/1)

Take-Home Midterm Exam Report

Bangkok CCTV Vehicle Detection

Name: <NAME SURNAME>

Student ID: <STUDENT ID>

Kaggle username: <KAGGLE USERNAME>

Source code / data / weights: <GitHub / Google Drive link>

Model weights (Google Drive): <Google Drive link>

Chapter 1: Introduction

The task is multi-class object detection of vehicles in Bangkok traffic-camera (CCTV) images. The images were scraped automatically every 30 minutes from the Bangkok Metropolitan Administration (BMA) traffic platform (cpudapp.bangkok.go.th/bmatraffic) between 25/08/2026 06:00 and 31/08/2026 21:00. After a quality screening, 20 cameras were kept. Every vehicle has to be localised with a bounding box and assigned to one of eight classes: Car, Motorcycle, Bus, Truck, Tuktuk, Van, Pickup and Songthaew.

Submissions are scored on Kaggle with **mAP@50**: the COCO-style average precision at IoU 0.50, computed with pycocotools for each class and averaged over the eight classes. Because every class has the same weight, rare classes such as Songthaew or Tuktuk affect the score as much as Car does.

Objectives of this work:

- explore and clean the data, and build a validation scheme that reflects the hidden test set;
- fine-tune a modern pretrained detector (YOLO26 from the Ultralytics framework) and evaluate it with the same metric as Kaggle;
- analyse the errors and describe how the model could be improved.

All code is in a single Jupyter notebook (`vehicle_detection.ipynb`) that runs end-to-end: EDA, cleaning, split, training, evaluation, visualisation and submission. No generative-AI / VLM API was used for prediction, so there is no API cost; training ran on a local laptop GPU (NVIDIA RTX 3050 Ti, 4 GB).

Chapter 2: Data Preparation

The analysis follows the data-preparation steps taught in the course (Week 2 slides and Lab1-Lab4): (1) examine the data set, (2) understand / narrow down the features and check for leakage, (3) handle outliers and skewness, and (4) build the train / validation split with a fixed random seed.

2.1 Examining the data set

Table 1: Dataset summary

Item	Value
Train images	2,991 (15 cameras)
Test images	1,013 (5 cameras); 997 are listed in <code>sample_submission.csv</code>
Image size	352 x 288 pixels (all images)
Annotated boxes (train)	27,396 in 2,889 images; 102 images have no vehicle
Boxes per image	mean 9.2, max 48
Box size (w x h)	median 13 x 20 px, 25% of boxes are smaller than 8 x 12 px
Submission format	id, image_id, class_id, confidence, x1, y1, x2, y2 (one row per box)

Table 2: Data dictionary of `train.csv` (27,396 rows x 7 columns)

Column	Type	Description	Missing
id	int	row id - identifier only, not a feature	0
image_id	str	<camera>_<YYYYMMDD>_<HHMMSS>.jpg; camera, date, hour and day/night are derived from it	0
class_id	int (nominal)	target class 0-7	0

x1, y1	int	top-left corner of the box [px]	0
x2, y2	int	bottom-right corner of the box [px]	0

Out-of-range values and miscodes. No missing values; all class ids are in 0-7; no inverted boxes and no box outside the image; every image_id exists on disk and all images are 352 x 288. Found: 25 exact duplicate boxes, 15 boxes with a side < 2 px, and 102 training images without any vehicle (kept as negative examples).

Target distribution

Table 3: Class distribution in the training set

ID	Class	Boxes	Share	Images containing class	Imbalance vs Car
0	Car	17,762	64.83%	2,712	1.0x
1	Motorcycle	6,563	23.96%	2,160	2.7x
2	Bus	482	1.76%	410	36.9x
3	Truck	1,356	4.95%	962	13.1x
4	Tuktuk	265	0.97%	226	67.0x
5	Van	354	1.29%	294	50.2x
6	Pickup	497	1.81%	408	35.7x
7	Songthaew	117	0.43%	59	151.8x

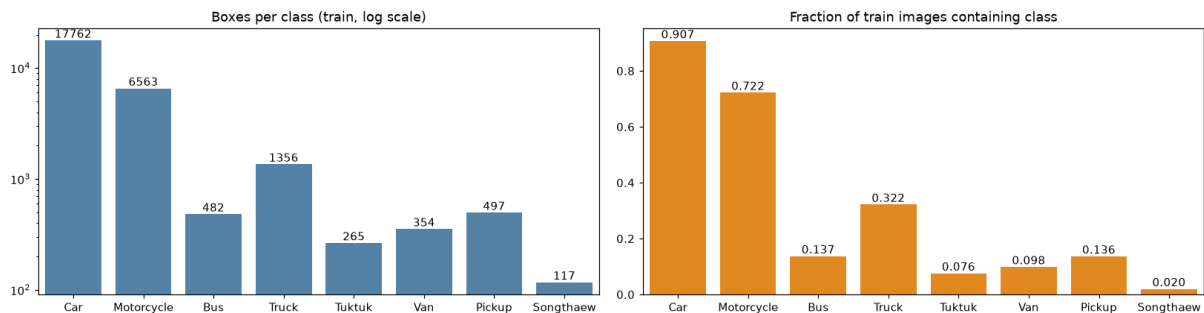


Figure 1: Boxes per class (log scale) and fraction of training images that contain each class

The target is heavily imbalanced: Car is 64.8% of all boxes while Songthaew is 0.43% (152 times fewer). Because mAP@50 averages the AP of the eight classes equally, the rare classes matter as much as Car.

2.2 Feature understanding, association with the target and leakage

Camera. Train uses cameras 1066, 12, 1407, 1426, 1427, 1437, 172, 180, 182, 214, 222, 229, 231, 232, 244; test uses 1068, 1072, 1192, 1439, 227. **The two sets are disjoint**, so the model must generalise to unseen viewpoints.

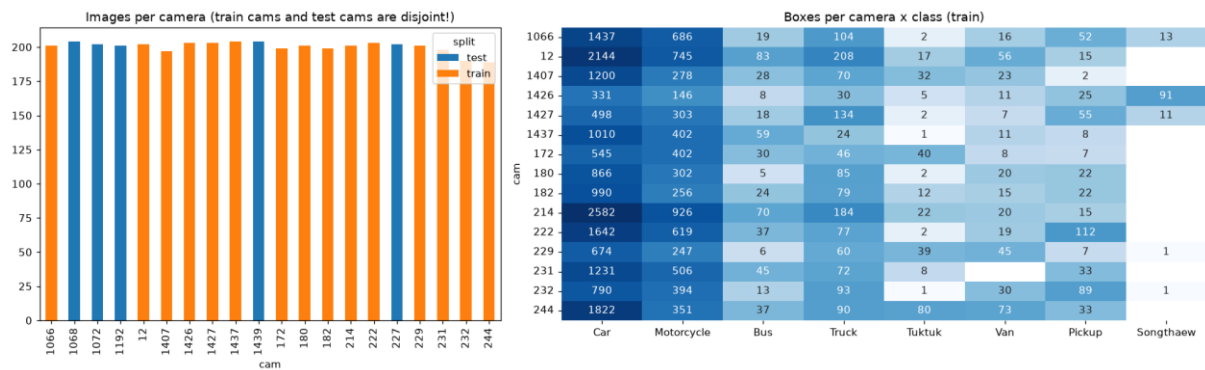


Figure 2: Images per camera (train vs test) and boxes per camera x class

Chi-square test of camera x class (slide appendix): $\chi^2 = 4,823$, $\text{dof} = 98$, $p < 1e-300$, Cramer's $V = 0.159$. H_0 (same class distribution for every camera) is rejected: the vehicle mix depends strongly on the camera. The standardised residuals show, for example, that 91 of the 117 Songthaews come from camera 1426 (residual +53), Pickups are over-represented at cameras 232 and 222 and Tuktuks at 244, 229 and 172.

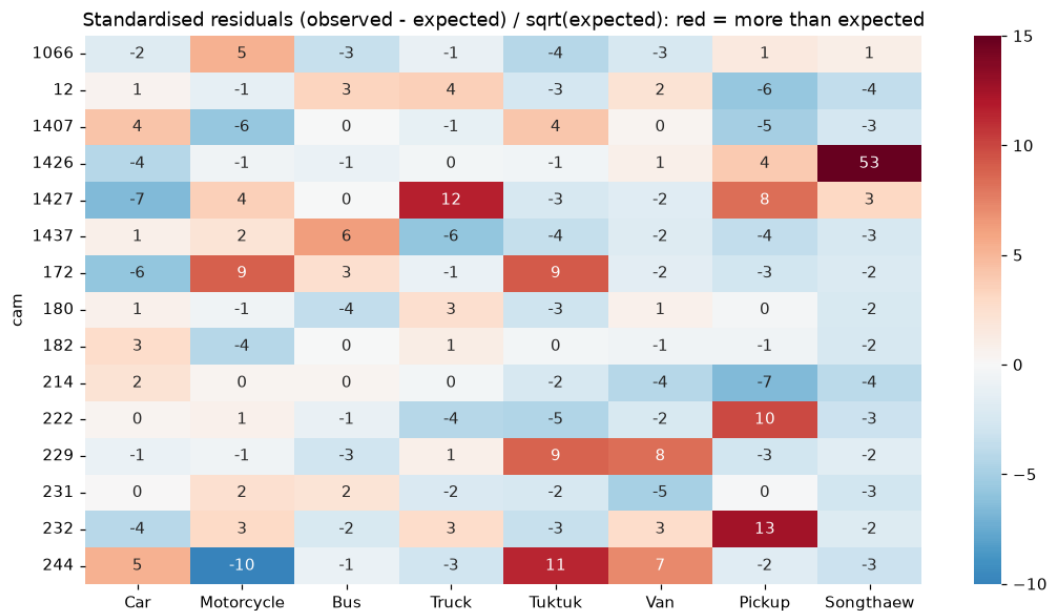


Figure 3: Standardised chi-square residuals, camera x class (red = more boxes than expected)

Time of day. Frames cover 06:00-21:00 every 30 minutes. Street lights keep night frames bright, so night was defined by clock time (after 18:30, about 17% of frames). The class mix also changes at night (χ^2 -square $p = 6.1e-33$, Cramer's $V = 0.078$); e.g. Songthaew rises from 0.27% of boxes by day to 1.53% at night and Tuktuk from 0.84% to 1.86%.

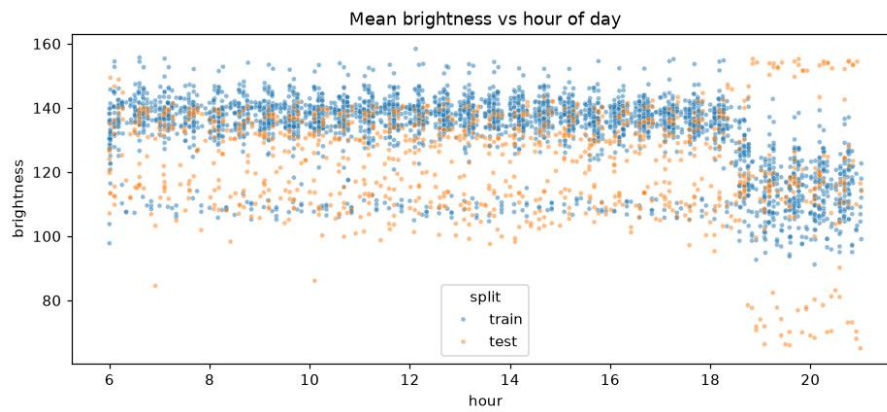


Figure 4: Mean brightness vs hour of day

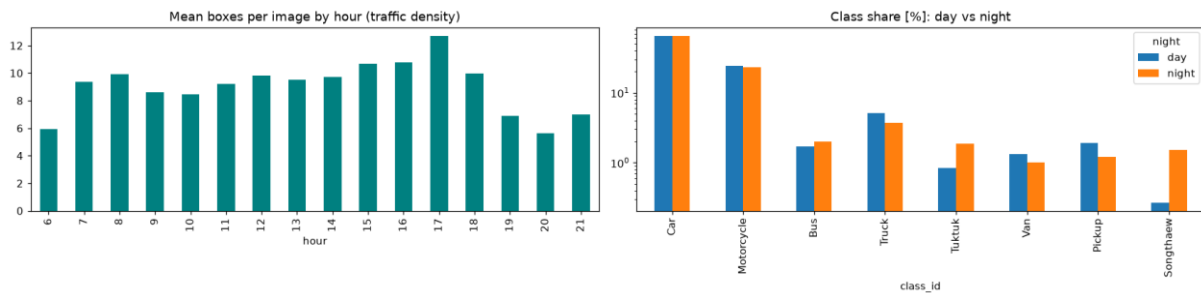


Figure 5: Traffic density by hour and class share by day / night

Leakage check. In Lab1, columns that leak information were removed. Here the leak is at the image level: consecutive frames of the same camera differ by a median of 14.5 grey levels, versus 53.6 for frames of different cameras (3.7x more similar). A random train / validation split would put almost identical scenes on both sides and measure memorisation instead of generalisation. Therefore **whole cameras are held out** for validation (Section 2.6).

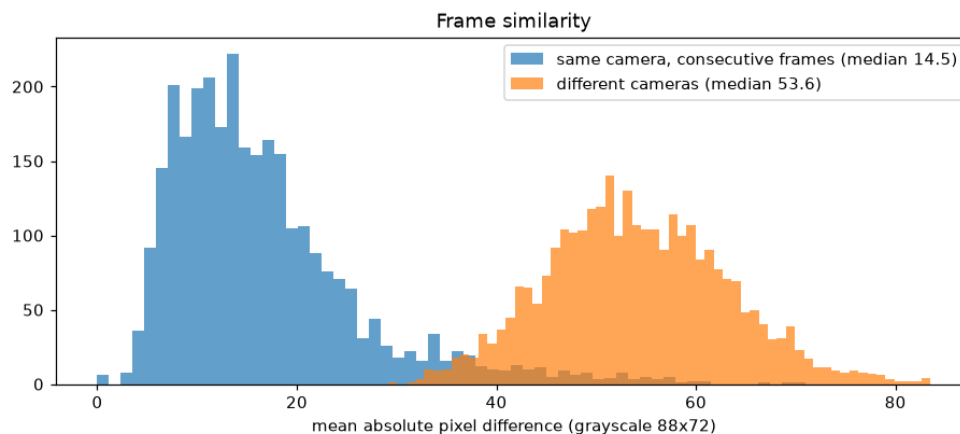


Figure 6: Frame similarity: same camera (consecutive frames) vs different cameras

Train vs test distribution. The test cameras have a similar share of night frames (17.6% vs 17.1%) and a slightly darker mean brightness (123 vs 132).

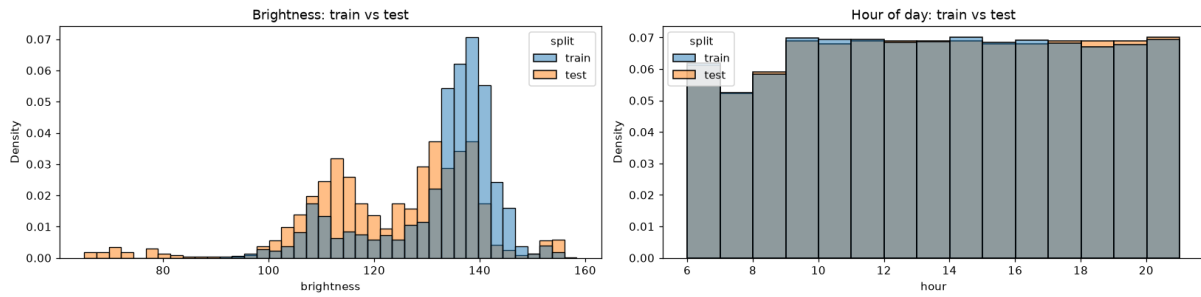


Figure 7: Brightness and hour-of-day distributions of train and test images

2.3 Outliers and skewness

Box sizes are tiny and strongly right-skewed: the box area has skew 7.05 and kurtosis 96.0; after a log transform the skew is 0.19 (Lab4 approach). This motivates upscaling the images (imgsz 640 -> 960) and a multi-scale detector.

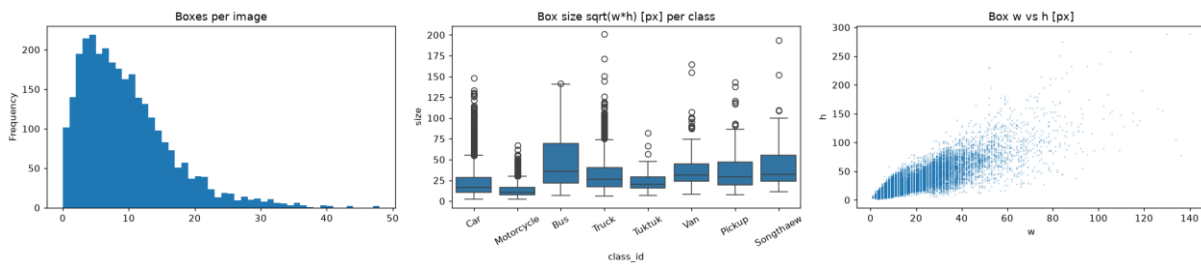


Figure 8: Boxes per image, box size per class and box width vs height

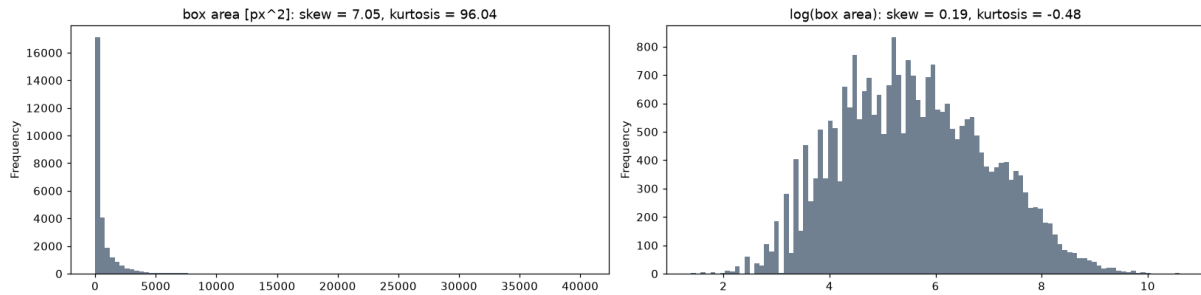


Figure 9: Box area before and after log transform

Table 4: Outliers per class (1.5 x IQR rule)

Class	Boxes	Median area	log-area IQR outliers	Median h/w	h/w IQR outliers
Car	17762	276	5	1.33	452
Motorcycle	6563	119	14	2.56	58
Bus	482	1272	0	1.8	6
Truck	1356	670	4	1.78	13
Tuktuk	265	420	1	1.93	0
Van	354	990	4	1.6	2
Pickup	497	840	0	1.42	1
Songthaew	117	1054	0	1.54	14

The extremes were inspected visually before deciding what to do. The largest boxes are real vehicles very close to camera 1426 (6 boxes are almost full height) and are kept. The extreme aspect ratios are

annotation errors: slivers such as 1×8 or 2×16 px with no vehicle inside (195 boxes have a side ≤ 2 px). Boxes with a side < 2 px are removed; truncating coordinates, as done for numeric features in Lab4, is not meaningful for boxes.



Figure 10: Most extreme boxes: largest area (top), tallest (middle), widest (bottom)

2.4 Visual inspection



Figure 11: One daytime frame per training camera with ground-truth boxes



Figure 12: Test cameras (day and night); these viewpoints never appear in training

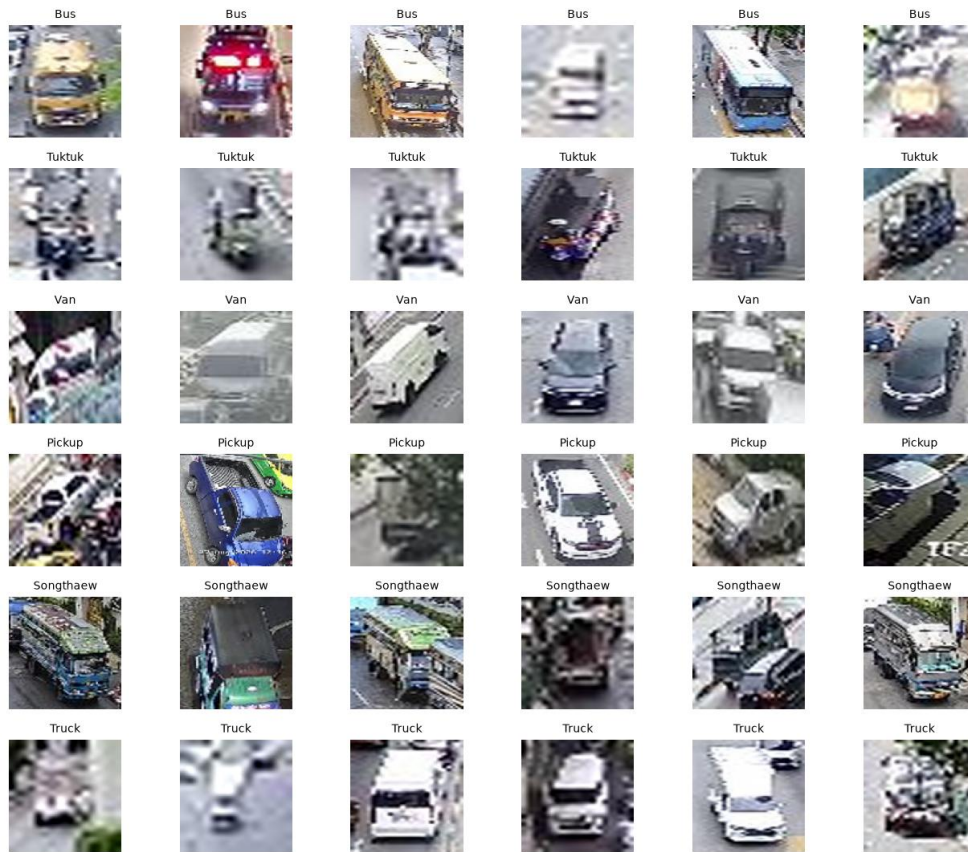


Figure 13: Examples of rare classes (crops). Van / Pickup / Truck are hard to separate even for a human

Submission ids differ from file names. sample_submission.csv uses long ids (dataset_<cam>_annotated_coco1.0_<cam>_<Thai location name>_<date>_<time>.jpg); they were mapped to test files by (camera, timestamp). 16 test images are not listed in sample_submission (not scored).

2.5 Data cleaning

Cleaning is applied to the **training labels only**. Validation is always scored against the raw labels, the same way Kaggle scores the raw test labels.

Table 5: Data-cleaning log

Step	Count
raw boxes	27396
boxes on missing images	0
boxes outside image (clipped)	0
exact duplicates removed	25
tiny boxes (< 2 px) removed	15
near-duplicates removed (same class, IoU > 0.9)	9
class conflicts kept (different class, IoU > 0.9)	280
clean boxes	27347
background images (no boxes)	102

280 pairs of boxes overlap with IoU > 0.9 but have different classes (mostly Car vs Truck and Bus vs Truck): the same vehicle was labelled twice. They were kept because the test labels probably contain the same pattern, and a model that outputs both classes at lower confidence can still match both.

2.6 Train / validation split and dataset construction

To mimic the unseen-camera test set and avoid the leakage shown in 2.2, **4 whole cameras (1066, 1427, 1437, 244) were held out for validation**. As a stratification over cameras, the notebook tries all 1,365 combinations of 4 cameras out of 15 and keeps the one where every class has at least 10 validation boxes and the validation share of each class is closest to 4/15. The random seed is fixed (42) everywhere.

Table 6: Camera-held-out split after cleaning: 2,194 train / 797 val images (class proportions compared as in Lab3)

Class	Train boxes	Val boxes	Val share	Train %	Val %
Car	12969	4765	26.9%	64.87%	64.79%
Motorcycle	4808	1741	26.6%	24.05%	23.67%
Bus	346	133	27.8%	1.73%	1.81%
Truck	1002	352	26.0%	5.01%	4.79%
Tuktuk	180	85	32.1%	0.90%	1.16%
Van	247	107	30.2%	1.24%	1.45%
Pickup	347	148	29.9%	1.74%	2.01%
Songthaew	93	24	20.5%	0.47%	0.33%

Repeat-factor sampling (RFS). mAP weights all classes equally, so images that contain rare classes are repeated during training (LVIS-style): for class c with image frequency f_c , $r_c = \max(1, \sqrt{t / f_c})$ with $t = 0.15$, and each image is repeated $r = \max_c r_c$ times with stochastic rounding. Songthaew images are repeated about 2.8x, Tuktuk 1.4x and Van 1.2x. Repeats are only added to the training list, never to validation.

Labels were converted to the YOLO format (class x_center y_center width height, normalised) and the image lists were written as train.txt / val.txt / full.txt, which Ultralytics reads directly.

2.7 Summary: findings and decisions

Table 7: EDA findings and the resulting decisions

Finding	Decision
Test cameras never appear in training; same-camera frames are near-duplicates	Hold out whole cameras for validation
Class mix depends on camera and time (chi-square $p \sim 0$)	Every class must be present in val; RFS for rare classes
Car : Songthaew = 152 : 1	Repeat-factor sampling; per-class AP analysis
Tiny, right-skewed box sizes	Upscale to imgsz 960; multi-scale detector
Sliver boxes are label noise; duplicates	Remove side < 2 px and exact / near duplicates
17% night frames in both train and test	Report day / night mAP; HSV augmentation

Chapter 3: Model

3.1 Choice of model

The course slides (Week 5, Object Detection) describe the change from two-stage CNNs (Faster R-CNN) to one-stage YOLO detectors, transformer detectors (DETR / RT-DETR) and, most recently, end-to-end NMS-free YOLO26. The course example notebook fine-tunes YOLOv8 with Ultralytics. This work uses the newest model in that family, **YOLO26**, fine-tuned from COCO-pretrained weights. YOLO11 and RT-DETR can be selected with a single config line for comparison.

Table 8: YOLO26s summary

Property	YOLO26s (used)
Framework	Ultralytics 8.4.163, PyTorch 2.8.0 + CUDA 12.6
Parameters / FLOPs	9.95 M parameters, 22.8 GFLOPs at 640 (8-class head)
Pretraining	COCO 2017 detection (80 classes, ~118k images)
Backbone	CSP-style CNN with C3k2 blocks, SPPF (spatial pyramid pooling) and C2PSA (attention) block
Neck	PAN/FPN feature pyramid, fuses strides 8 / 16 / 32 (important for small vehicles)
Head	Anchor-free, end-to-end (one-to-one) head: no NMS at inference; no DFL
Transfer	4 of 8 class rows of the head (car, motorcycle, bus, truck) were initialised from the matching COCO classes

YOLO is a one-stage detector: a single CNN pass predicts, for every location of three feature maps, a box and class scores. YOLO26 trains with a one-to-one assignment so each object gets a single prediction, which removes the non-maximum-suppression step, and it drops the Distribution Focal Loss. According to the Ultralytics documentation it also uses progressive loss balancing (ProgLoss) and small-target-aware label assignment (STAL), which suits this dataset because most vehicles are tiny.

3.2 Training setup

Table 9: Training configuration

Hyper-parameter	Value
Input size	960 (images are upscaled from 352 x 288 with letterbox)
Epochs / early stopping	40 max, patience 30; the best-val-mAP checkpoint is kept
Batch size	4 (limited by 4 GB VRAM)
Optimizer	Ultralytics "auto" -> AdamW (lr 0.000833, momentum 0.9, weight decay 0.0005), cosine LR schedule, warm-up 3 epochs
Augmentation	Mosaic (off for the last 10 epochs), random scale 0.5, translate 0.1, horizontal flip 0.5, HSV jitter (h 0.015, s 0.7, v 0.4), random erasing 0.4
Class balance	Repeat-factor sampling (Section 2.4)
Seed	42, deterministic=True
Hardware	NVIDIA GeForce RTX 3050 Ti Laptop GPU (4 GB), 32 GB RAM, Windows 11

3.3 Inference and post-processing

- Confidence threshold 0.001: low-confidence boxes only extend the precision-recall curve and never reduce AP, so a very low threshold gives the highest mAP.
- max_det = 100 boxes per image (pycocotools keeps at most 100 per image and class).
- Boxes are written in absolute pixel coordinates; degenerate boxes are removed; ids are mapped back to the long sample_submission ids; every scored image appears at least once.

3.4 Evaluation

The notebook re-implements the Kaggle metric with pycocotools (COCOeval, IoU = 0.50, area = all, maxDets = 100) on the held-out cameras, using the raw labels. This is the number used to choose models. Ultralytics' own mAP50 is a bit higher because it uses the cleaned labels.

Chapter 4: Results

4.1 Experiments (validation = held-out cameras)

Table 10: Experiment summary

Run	Description	Val mAP@50	Kaggle public
yolo26s_640	YOLO26s, imgsz 640, batch 8, RFS; 100 ep max, early-stopped at 60 (best ep 30)	0.4666	-
yolo26s_640_tta	same weights + test-time augmentation (flips + multi-scale)	0.4919	-
yolo26s_960	YOLO26s, imgsz 960, batch 4, RFS; 40 ep (best ep 22)	0.4958	-
yolo26s_960_tta	yolo26s_960 weights + TTA	0.5247	-
yolo26s_960_full_tta	final: same settings trained on all 15 cameras (25 ep, last epoch) + TTA; val cams are inside training, so no unbiased val score	-	-

4.2 Per-class AP of the best model

Table 11: Per-class AP@50 on validation cameras (yolo26s_960_tta)

Class	Car	Motorcycle	Bus	Truck	Tuktuk	Van	Pickup	Songthaew
AP@50	0.901	0.800	0.699	0.588	0.448	0.337	0.315	0.109

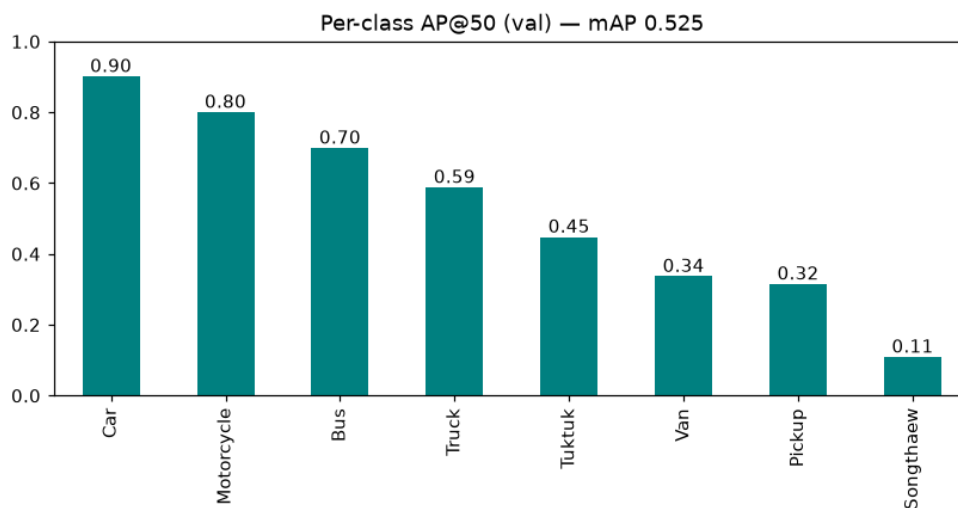


Figure 14: Per-class AP@50 (validation)

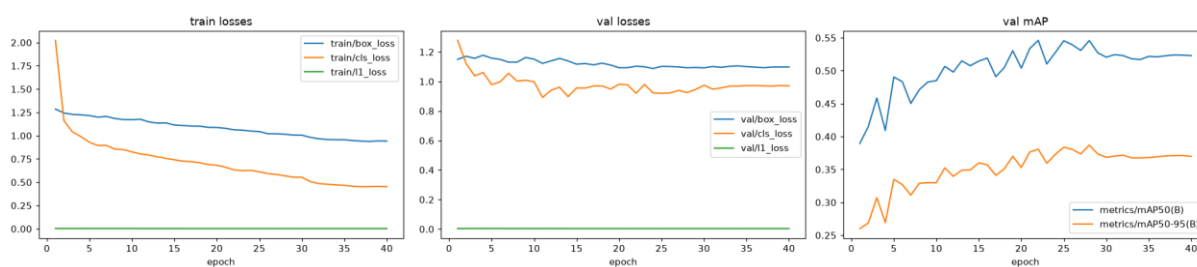


Figure 15: Training curves (losses and validation mAP)

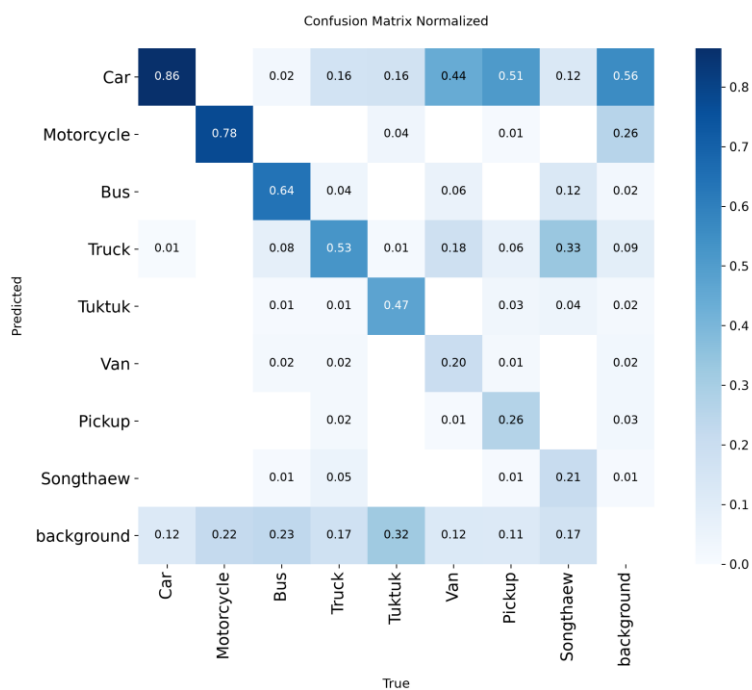


Figure 16: Normalised confusion matrix on the validation cameras

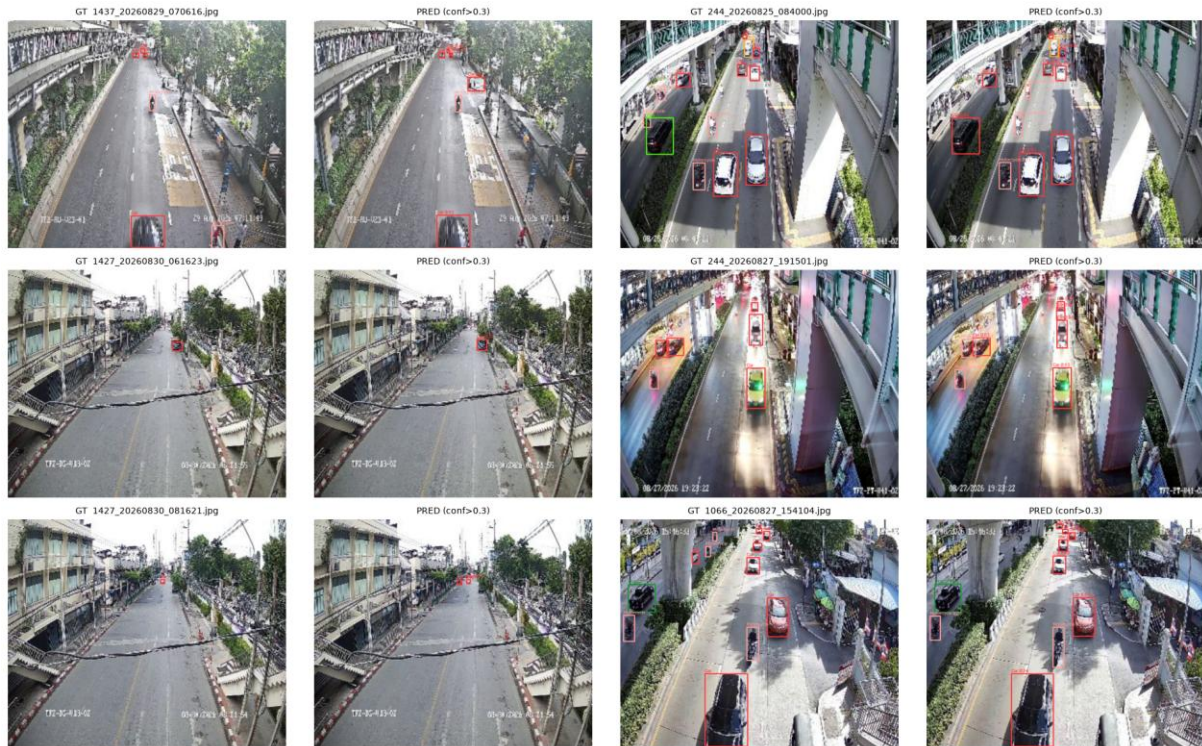


Figure 17: Ground truth (left) vs prediction with confidence > 0.3 (right) on validation cameras

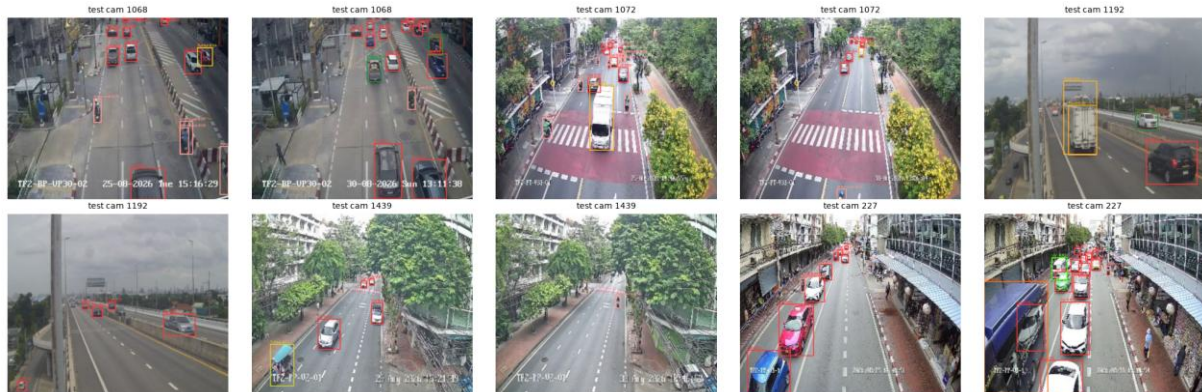


Figure 18: Predictions on the unseen test cameras

4.3 Kaggle submission

Final submission file: submissions/yolo26s_960_full_tta.csv (fallback: submissions/yolo26s_960_tta.csv).

Public leaderboard mAP@50: -.

Model weights: weights/yolo26s_960_full.pt (final, all cameras) and weights/yolo26s_960.pt (trained without the 4 validation cameras).

[Insert screenshot of the Kaggle submission page / leaderboard here]

Chapter 5: Discussion

5.1 Error analysis by class

- **Car (0.90) and Motorcycle (0.80)** are detected well: they are frequent and look distinctive.

- **Van (0.34), Pickup (0.32) and Songthaew (0.11)** are the main weakness. The confusion matrix shows that 44% of Vans and 51% of Pickups are predicted as Car, and 33% of Songthaews as Truck. At 15-25 px these vehicles differ only by a few pixels (roof shape, open cargo bed, rear bench).
- **Label noise** makes this harder: the crops in Chapter 2 contain SUVs labelled as Van and vans labelled as Truck, and 280 vehicles carry two boxes with different classes (mostly Car + Truck). One of the two boxes can never be matched by a single prediction, which caps the achievable AP.
- **Songthaew generalises poorly** because 91 of its 117 training boxes come from one camera (1426), so the model learns that camera's appearance rather than the vehicle.
- **Tuktuk** is often missed completely (32% predicted as background): tuktuks are small, partially covered by motorcycles and look like a motorcycle with a roof.

5.2 Error analysis by condition and camera

- Day vs night (960 + TTA): 0.537 vs 0.494. Headlight glare and motion blur reduce the visible shape.
- Per validation camera: 0.52 (cam 244, 1427) to 0.69 (cam 1437). Accuracy depends strongly on the viewpoint, so the unseen test cameras are the biggest source of uncertainty.
- Overfitting to the training cameras: validation mAP peaks early (epoch 30 at 640, epoch 22 at 960) and then slowly decreases while the training loss keeps going down. Early stopping on held-out cameras was therefore essential.

5.3 What helped

- Camera-held-out validation: gives an honest estimate for unseen cameras (a random split would mix near-identical frames of the same camera).
- Higher input resolution (640 -> 960): +0.029 mAP, mainly Truck (+0.10) and Tuktuk (+0.05).
- Test-time augmentation: +0.025 at 640 and +0.029 at 960; it helps the rare classes most (Van, Pickup, Bus).
- Very low confidence threshold (0.001) so that the precision-recall curve is complete. Keeping 300 instead of 100 boxes per image changed the score by only +0.001.

5.4 How to improve

- Larger backbones (YOLO26m/l, RT-DETR) and imgsz 1280 on a bigger GPU (Colab / Kaggle); only a 4 GB GPU was available here.
- A second-stage classifier (or a VLM) that re-classifies crops of the confusable classes Car / Van / Pickup / Truck / Songthaew at higher resolution, emitting the alternative classes with lower scores.
- Ensembling several models / resolutions with Weighted Boxes Fusion.
- Semi-supervised domain adaptation: pseudo-label the unlabelled test cameras with high-confidence predictions and fine-tune.
- Copy-paste augmentation of rare vehicles into other cameras, and relabelling the noisy Van / Truck / duplicate boxes.
- Cross-validation over several camera folds to choose hyper-parameters more reliably than a single 4-camera split.

Chapter 6: Conclusion

A YOLO26s detector pretrained on COCO was fine-tuned to detect eight vehicle classes in low-resolution Bangkok CCTV images. The EDA showed that the test cameras never appear in training, the objects are tiny and the classes are strongly imbalanced; the pipeline was designed around these facts with a

camera-held-out validation split, repeat-factor sampling of rare classes, upscaled training (imgsz 960) and test-time augmentation.

On four held-out cameras the model reaches $mAP@50 = 0.525$ (Car 0.90, Motorcycle 0.80, Bus 0.70), up from 0.467 for the 640 baseline. The final model is retrained on all 15 cameras with the same settings. The remaining errors come mostly from fine-grained confusion between Car, Van, Pickup, Truck and Songthaew and from viewpoint changes between cameras, which a second-stage classifier, larger models and domain adaptation could address.

The whole pipeline is reproducible from a single notebook with fixed seeds, pinned package versions and the submitted weights.

Appendix: How to reproduce

- `conda create -n dsde python=3.12; pip install torch==2.8.0 torchvision==0.23.0 --index-url https://download.pytorch.org/whl/cu126; pip install -r requirements.txt`
- `kaggle competitions download -c 2110531-dsde-2026-1 -p data/raw`, then unzip into `data/raw`
- Open `vehicle_detection.ipynb` from the `midterm/` folder and run all cells. The settings are in the Config cell (or set environment variables, e.g. `MODEL`, `IMGSZ`, `EPOCHS`, `FULL_TRAIN=1`).
- To reproduce the submission without training: `SKIP_TRAIN=1 WEIGHTS=<path to submitted .pt>` (and the same `IMGSZ` / `TTA` as in the submission `.json` file).